

0. 方法核心一句话

你的方法可以叫：

PAIR: Perception-Action Interface Representation

它要学的是一个连接 **perceptual space** 和 **action space** 的中间表征：

$$\mathcal{M}_P \rightarrow \mathcal{M}_{PA} \rightarrow \mathcal{M}_A$$

其中：

- $\mathcal{M}_P$ ：由 vision tokens 和 instruction embeddings 构成的 task-conditioned perceptual space；
- $\mathcal{M}_A$ ：动作轨迹 / action chunk 所在的 action space；
- $\mathcal{M}_{PA}$ ：你要学的 perception-action interface space。

你的 bridge tokens 从一开始的 perception tokens 中读信息，然后被 action autoencoder 的 latent 约束，再用来初始化 VLA-Adapter 原本空的 action states。

最终链条是：

$$[V_0; T_0] \rightarrow Z_B \rightarrow Q_a^0 \rightarrow \hat{a}_{t:t+H}$$

其中 Z_B 是你学到的 **perception-action bridge representation**。

1. VLA-Adapter 里的关键维度

以 LIBERO 默认配置为例，VLA-Adapter 的 constants 里：

$$H = \text{NUM_ACTIONS_CHUNK} = 8$$

$$d_a = \text{ACTION_DIM} = 7$$

$$d_p = \text{PROPRIO_DIM} = 8$$

并且 action/proprio normalization 默认是 `BOUNDS_Q99`。这些都在 VLA-Adapter 的 constants 文件里定义。 [GitHub](#)

VLA-Adapter 的 `L1RegressionActionHead` 默认：

$$D = \text{hidden_dim} = 4096$$

它内部会先构造一个全 0 的 action hidden state：

$$\text{cond_actions_hidden_states} \in \mathbb{R}^{B \times (d_a H) \times D}$$

对 LIBERO 来说：

$$d_a H = 7 \times 8 = 56$$

所以是：

$$[B, 56, 4096]$$

然后它 reshape 成：

$$[B, H, d_a D] = [B, 8, 7 \times 4096] = [B, 8, 28672]$$

再送进 `MLPResNet`。源码中确实是先创建 `(batch_size, self.action_dim * NUM_ACTIONS_CHUNK, self.hidden_dim)` 的零张量，再 reshape 成 `(batch_size, NUM_ACTIONS_CHUNK, -1)`。 [GitHub](#)

所以你要初始化的不是 raw action：

$$[B, 8, 7]$$

而是 VLA-Adapter action head 的内部初始 hidden state：

$$[B, 56, 4096]$$

或者 reshape 后：

$$[B, 8, 28672]$$

2. 最终固定维度

我们最终使用：

$$H = 8$$

$$d_a = 7$$

$$D = 4096$$

$$K = 8$$

$$d_z = 16$$

$$d_b = 512$$

解释一下：

符号	值	含义
H	8	action chunk 长度
d_a	7	每步 action 维度
D	4096	VLA hidden dim
K	8	bridge/action latent token 数
d_z	16	action AE latent dim
d_b	512	bridge query 内部降维后的维度

为什么 $K = 8$ ：

$$K = H$$

这样每个 bridge token 可以自然对应 action chunk 的一个时间 step，后面初始化 action state 时最清楚。

为什么 $d_z = 16$ ：

$$8 \times 16 = 128$$

action chunk 原始维度是：

$$8 \times 7 = 56$$

128 维 latent 足够表达 action chunk，但不会像 $8 \times 32 = 256$ 那样过宽。

3. Stage 1: Action Autoencoder

Action AE 的作用是：学习一个 action-side latent teacher。

它不是主贡献，也不是 inference 时用的模块。它只是为了提供：

$$A = E_A(a_{t:t+H})$$

其中：

$$A \in \mathbb{R}^{B \times 8 \times 16}$$

后面用它监督 bridge tokens：

$$Z_B^{align} \approx A$$

3.1 输入

输入是 normalized action chunk：

$$a \in \mathbb{R}^{B \times 8 \times 7}$$

注意：这里要用和 VLA-Adapter action training 一致的 normalized action。LIBERO 默认 action normalization 是 `BOUNDS_Q99`。[GitHub](#)

3.2 Encoder 结构

你提议 encoder 用 4 个 self-attention block, 我认为最终就这么定。

Encoder:

$$a \in [B, 8, 7]$$

先投影:

$$x_0 = W_{in}a + p$$

其中:

$$W_{in} : 7 \rightarrow 64$$

$$x_0 \in [B, 8, 64]$$

然后过 4 个 self-attention blocks:

$$x_{l+1} = \text{SA-Block}(x_l), \quad l = 0, 1, 2, 3$$

最后投影到 latent:

$$A = W_z x_4$$

其中:

$$W_z : 64 \rightarrow 16$$

所以:

$$A \in [B, 8, 16]$$

图示:

```
Action chunk
[B, 8, 7]
  ↓ Linear(7 → 64) + temporal pos
[B, 8, 64]
  ↓ 4 × Self-Attention Block
[B, 8, 64]
  ↓ Linear(64 → 16)
Action latent A
[B, 8, 16]
```

3.3 Decoder 结构

Decoder 不需要太强，建议 2 个 self-attention block。

$$A \in [B, 8, 16]$$

先升维：

$$y_0 = W_{dec}A + p_{dec}$$

其中：

$$W_{dec} : 16 \rightarrow 64$$

$$y_0 \in [B, 8, 64]$$

然后：

$$y_{l+1} = \text{SA-Block}(y_l), \quad l = 0, 1$$

最后输出重建 action：

$$\tilde{a} = W_{out}y_2$$

其中：

$$W_{out} : 64 \rightarrow 7$$

$$\tilde{a} \in [B, 8, 7]$$

图示：

```
Action latent A
[B, 8, 16]
  ↓ Linear(16 → 64) + temporal pos
[B, 8, 64]
  ↓ 2 × Self-Attention Block
[B, 8, 64]
  ↓ Linear(64 → 7)
Reconstructed action
[B, 8, 7]
```

3.4 AE loss

$$\mathcal{L}_{AE} = \|\tilde{a} - a\|_1$$

只在 normalized action space 上算 L1。

训练完成后：

$$E_A$$

冻结。

Stage 2 只用：

$$A = \text{sg}(E_A(a))$$

不再使用 decoder D_A 。

4. Stage 2: Bridge Query 从初始 perceptual tokens 读信息

这里我们已经修正过逻辑：不能拿 action head 后面的 task hidden states 来生成 bridge token，因为你的方法是要用 bridge token 初始化 action state，所以 bridge source 必须在 action state 之前。

因此，bridge query 的输入应该是最开始的 perceptual tokens：

$$X_P^0 = [V_0; T_0]$$

其中：

- V_0 : vision backbone / vision projector 后的 vision tokens;
- T_0 : instruction token embeddings / contextual instruction embeddings;
- 二者都在 VLA hidden dim $D = 4096$ 或可投影到同一维度。

所以：

$$X_P^0 \in \mathbb{R}^{B \times N \times D}$$

其中：

$$N = N_v + N_t$$

4.1 Bridge query 内部降维

因为直接在 4096 维做 cross-attention 比较重，所以我们先降维：

$$X_P' = W_d X_P^0$$

其中：

$$W_d : 4096 \rightarrow 512$$

所以：

$$X_P' \in [B, N, 512]$$

4.1 Bridge query 内部降维

因为直接在 4096 维做 cross-attention 比较重，所以我们先降维：

$$X'_P = W_d X_P^0$$

其中：

$$W_d : 4096 \rightarrow 512$$

所以：

$$X'_P \in [B, N, 512]$$

4.2 Learnable bridge queries

定义 8 个 learnable bridge queries：

$$Q_B \in \mathbb{R}^{8 \times 512}$$

它们 cross-attend 到 perception tokens：

$$Z'_B = \text{CrossAttn}(Q_B, X'_P, X'_P)$$

输出：

$$Z'_B \in [B, 8, 512]$$

这里 Z'_B 是低维 bridge representation。

图示：

```
Initial perceptual tokens
X_P^0 = [V_0; T_0]
[B, N, 4096]
  ↓ Linear(4096 → 512)
X_P'
[B, N, 512]

Learnable bridge queries
Q_B
[8, 512]
  ↓ CrossAttn(Q_B, X_P', X_P')
Z_B'
[B, 8, 512]
```

5. Action-manifold alignment 分支

Stage 2 训练时, ground-truth action chunk 经过 frozen action encoder:

$$A = \text{sg}(E_A(a))$$

其中:

$$A \in [B, 8, 16]$$

Bridge token 走一个 alignment projection:

$$Z_B^{\text{align}} = W_{\text{align}} Z'_B$$

其中:

$$W_{\text{align}} : 512 \rightarrow 16$$

所以:

$$Z_B^{\text{align}} \in [B, 8, 16]$$

然后和 action latent 对齐:

$$\mathcal{L}_{\text{align}} = \frac{1}{8} \sum_{h=1}^8 \left[1 - \cos(Z_{B,h}^{\text{align}}, A_h) \right]$$

这一步的意义是:

▮ Z'_B 虽然从 perceptual tokens 中抽出来, 但它的几何结构被 action latent manifold 约束。

也就是:

$$\mathcal{M}_P \rightarrow \mathcal{M}_{PA} \approx \mathcal{M}_A^z$$

6. Action-state initialization 分支

Bridge tokens 还要用来初始化 VLA-Adapter 原本空的 action hidden states。

先把低维 bridge token 升回 VLA hidden dim:

$$Z_B = W_u Z'_B$$

其中:

$$W_u : 512 \rightarrow 4096$$

所以:

$$Z_B \in [B, 8, 4096]$$

6.1 生成 $[B, 8, 7, 4096]$ 的 bridge init

VLA-Adapter 的 action hidden state 在 reshape 前是：

$$[B, 56, 4096]$$

可以理解成：

$$[B, 8, 7, 4096]$$

也就是：

- 8 个未来时间步；
- 每个时间步 7 个 action-dim slot；
- 每个 slot 是 4096 维。

我们用每个 step 的 bridge token $Z_{B,h} \in [4096]$ 生成 7 个 action-dim slots。

具体做法：

$$\tilde{Z}_B = \text{LN}(Z_B)$$

然后对每个 action 维度 $j = 1, \dots, 7$ ：

$$u_{h,j} = s_j \odot \tilde{Z}_{B,h}$$

其中：

$$s_j \in \mathbb{R}^{4096}$$

所以：

$$u_{h,j} \in \mathbb{R}^{4096}$$

把 7 个 slot 堆起来：

$$U_h = [u_{h,1}, u_{h,2}, \dots, u_{h,7}]$$

整体：

$$U \in [B, 8, 7, 4096]$$

reshape：

$$U \rightarrow [B, 56, 4096]$$

这个 U 就是：

$$x_0^{\text{bridge}}$$

6.2 为什么用 $s_j \odot Z$ 这种 gate

因为如果直接做：

$$Z_B \rightarrow [B, 8, 7 \cdot 4096]$$

的大线性层，参数量巨大：

$$4096 \times (7 \cdot 4096) \approx 117M$$

太重，而且不公平。

而使用：

$$u_{h,j} = s_j \odot \text{LN}(Z_{B,h})$$

参数量只有：

$$7 \times 4096$$

非常轻。

它的含义是：

每个 action 维度从同一个 bridge token 里选择不同的 feature channels。

位置、旋转、gripper 这些 action dimensions 不应该完全共享同一份初始化，所以给每个 action dimension 一个单独的 feature-wise gate。

是否需要 sigmoid / tanh?

我建议第一版用 **tanh gate**，因为 VLA-Adapter 自己就用了类似：

$$r = \tanh(g)$$

你可以写：

$$x_0 = x_0^{base} + \alpha \tanh(g) \cdot x_0^{bridge}$$

7. Stage 2 总 loss

Action prediction loss:

$$\mathcal{L}_{BC} = \|\hat{a} - a\|_1$$

Bridge alignment loss:

$$\mathcal{L}_{align} = \frac{1}{8} \sum_{h=1}^8 \left[1 - \cos(Z_{B,h}^{align}, \text{sg}(E_A(a)_h)) \right]$$

总 loss:

$$\mathcal{L} = \mathcal{L}_{BC} + \lambda \mathcal{L}_{align}$$

建议第一版:

$$\lambda = 0.05$$

可以做 warmup:

$$0 \rightarrow 0.05$$

例如前 5% 或 10% steps 线性升上去。

8. 训练流程

Stage 1: Action AE pretraining

只训练 action AE。

输入：

$$a \in [B, 8, 7]$$

输出：

$$\tilde{a} \in [B, 8, 7]$$

loss：

$$\mathcal{L}_{AE} = \|\tilde{a} - a\|_1$$

训练完保存 encoder E_A 。

推荐配置：

```
Action AE:  
input: [B, 8, 7]  
hidden: [B, 8, 64]  
encoder: 4 self-attention blocks  
latent: [B, 8, 16]  
decoder: 2 self-attention blocks  
output: [B, 8, 7]
```

Stage 2: VLA-Adapter + PAIR bridge

训练主 policy。

每个 batch:

1. 从初始 vision tokens 和 instruction embeddings 得到:

$$X_P^0 = [V_0; T_0]$$

2. Bridge query 生成:

$$Z'_B \in [B, 8, 512]$$

3. Alignment branch:

$$Z_B^{align} \in [B, 8, 16]$$

$$A = \text{sg}(E_A(a)) \in [B, 8, 16]$$

算:

$$\mathcal{L}_{align}$$

4. Init branch:

$$Z_B \in [B, 8, 4096]$$

$$x_0^{bridge} \in [B, 56, 4096]$$

5. 替换/增强原始 action init:

$$x_0 = x_0^{base} + \alpha x_0^{bridge}$$

6. 继续走 VLA-Adapter 原 action head, 输出:

$$\hat{a} \in [B, 8, 7]$$

7. 算:

$$\mathcal{L}_{BC} + \lambda \mathcal{L}_{align}$$

9. 推理流程

推理时没有 ground-truth action, 因此 action AE 不用。

只走:

$$[V_0; T_0] \rightarrow Z'_B \rightarrow Z_B \rightarrow x_0^{bridge} \rightarrow x_0 \rightarrow \hat{a}$$

即:

```
image + instruction
  ↓ vision backbone / text embedding
initial perceptual tokens [V_0; T_0]
  ↓ bridge queries
bridge tokens Z_B
  ↓ action-state init
initialized action states
  ↓ VLA-Adapter action head
predicted action chunk [B, 8, 7]
```

10. 最终设计的几个关键原则

原则 1: Bridge source 必须在 action token 之前

不用后面 action head 里的 task hidden states 作为主 source。

正确是：

$$[V_0; T_0] \rightarrow Z_B \rightarrow \text{action init}$$

因为你是要初始化 action state，所以 bridge 必须来自 initial perceptual tokens。

原则 2: Bridge token 不直接对齐 raw action

不做：

$$Z_B \approx a$$

而是：

$$Z_B^{\text{align}} \approx E_A(a)$$

即对齐 action AE latent。

原则 3: Action AE 只作为 teacher

Stage 2 不用 action decoder D_A 。

不做 shared decoder：

$$D_A(Z_B) \rightarrow a$$

只做版本 2：

$$W_{\text{align}} Z_B \approx E_A(a)$$

Stage 1:

$$A = E_A(a), \quad \tilde{a} = D_A(A)$$

$$\mathcal{L}_{AE} = \|\tilde{a} - a\|_1$$

Stage 2:

$$X_P^0 = [V_0; T_0]$$

$$X'_P = W_d X_P^0$$

$$Z'_B = \text{CrossAttn}(Q_B, X'_P, X'_P)$$

$$Z_B^{align} = W_{align} Z'_B$$

$$A = \text{sg}(E_A(a))$$

$$\mathcal{L}_{align} = \frac{1}{8} \sum_{h=1}^8 \left[1 - \cos(Z_{B,h}^{align}, A_h) \right]$$

$$Z_B = W_u Z'_B$$

$$x_{0,h,j}^{bridge} = s_j \odot \text{LN}(Z_{B,h})$$

$$x_0 = x_0^{base} + \alpha x_0^{bridge}$$

$$\hat{a} = D_{act}(x_0)$$

$$\boxed{\mathcal{L} = \|\hat{a} - a\|_1 + \lambda \mathcal{L}_{align}}$$

这就是最终方案。方法保持简单，但故事是高级的：**从 initial perceptual tokens 学一个 action-manifold-constrained bridge representation**，并用它初始化 VLA-Adapter 的空 action states，从而显式连接 perceptual space 和 action space。